CS315 · ALGORITHMS ANALYSIS & DESIGN

Midterm 451 Solutions

30/10/2023 · 1.5 hours · 25 marks

Muharib

01 Solution Q1: matrix chain

The idea: multiplying a `p × q` matrix by a `q × r` matrix costs `p·q·r` operations. We want the parenthesization with the smallest total. We fill the table from short chains to long chains.

```
m[i, j] = min over k of:   m[i, k] + m[k+1, j] + cost
```

```
cost = (rows of i) × (cols of k) × (cols of j)
```

1 Chains of two matrices

```
AB  =  4 × 5 × 8  =  160
```

```
BC  =  5 × 8 × 3  =  120
```

```
CD  =  8 × 3 × 7  =  168
```

2 Chains of three matrices

```
ABC:   A(BC)  =  0 + 120 + 4 × 5 × 3  =  180     ← min
       (AB)C  =  160 + 0 + 4 × 8 × 3  =  256
BCD:   B(CD)  =  0 + 168 + 5 × 8 × 7  =  448
       (BC)D  =  120 + 0 + 5 × 3 × 7  =  225     ← min
```

3 The full chain ABCD

```
A(BCD)    =  0 + 225 + 4 × 5 × 7    =  365
(AB)(CD)  =  160 + 168 + 4 × 8 × 7  =  552
(ABC)D    =  180 + 0 + 4 × 3 × 7    =  264     ← min
```

	A	B	C	D
A	0	160	180	**264**
B	—	0	120	225
C	—	—	0	168
D	—	—	—	0

```
Minimum operations = 264
```

```
Best order of multiplication:   ((A(BC))D)
```

The order comes from tracing the choices back: ABCD split as `(ABC)D`, and ABC split as `A(BC)`.

02 Solution Q2: hash table

Key	4	13	49	6	40	33
k mod 9	4	4	4	6	4	6

(A) Chaining: four keys go to cell 4 and two keys go to cell 6.

Index	Chain
0	∅
1	∅
2	∅
3	∅
4	4 → 13 → 49 → 40
5	∅
6	6 → 33
7	∅
8	∅

(B) Double hashing: probe number `j` is `(h1(k) + j·h2(k)) mod 9`.

Key	k mod 9	1 + (k mod 5)	Probes	Cell
4	4	5	4	4
13	4	4	4 → 8	8
49	4	5	4 → 0	0
6	6	2	6	6
40	4	1	4 → 5	5
33	6	4	6 → 1	1

0	1	2	3	4	5	6	7	8
49	33			4	40	6		13

13: h1 = 4 taken, h_2 = 1 + (13 mod 5) = 4 → (4 + 4) mod 9 = 8

49: h1 = 4 taken, h_2 = 1 + (49 mod 5) = 5 → (4 + 5) mod 9 = 0

40: h1 = 4 taken, h_2 = 1 + (40 mod 5) = 1 → (4 + 1) mod 9 = 5

33: h1 = 6 taken, h_2 = 1 + (33 mod 5) = 4 → (6 + 4) mod 9 = 1

03 Solution Q3: counting operations

Line	# operations
currentMax ← A[0]	2
Temp ← A[0]	2
for i ← 1 to n − 1 do	1 + n
if A[i] > currentMax then	2(n − 1)
currentMax ← A[i]	2(n − 1)
{ increment counter i }	2(n − 1)
return currentMax	1
Total	**7n**

2 + 2 + (1 + n) + 2(n − 1) + 2(n − 1) + 2(n − 1) + 1 = 7n

It is the lecture's table (`7n − 2`) plus two operations for the new line `Temp ← A[0]`. This count is for the worst case.

04 Solution Q4: True / False + MCQ

#	Answer	Reason
1	**a. True**	A character that occurs more often gets a shorter code.
2	**b. False**	Greedy does not give the optimum for every DP problem.
3	**c**	Dynamic programming stores the results of sub-problems and reuses them.
4	**b. False** *	The slide writes `T(MERGE)`, not $T(\frac{MERGE}{2})$.
5	**c. Other value** *	The time is `O(n)` when the activities are sorted by finish time.

About the two stars

Items 4 and 5 were repeated word for word in the 462 midterm, and I have a graded student paper from it: False for item 4 was marked correct, and for item 5 the highlighted answer is "Other value", with `O(n)` written beside it.

05 Solution Q5: LCS

We build a table whose rows are the letters of `X` and whose columns are the letters of `Y`, with a first row and column of zeros. Each cell is computed like this:

```
X[i] = Y[j]:   c[i, j] = c[i−1, j−1] + 1        diagonal + 1
```

```
X[i] ≠ Y[j]:   c[i, j] = max(top, left)
```

	⌀	A	B	C	A	D	F
⌀	0	0	0	0	0	0	0
A	0	1	1	1	1	1	1
C	0	1	1	2	2	2	2
B	0	1	2	2	2	2	2
A	0	1	2	2	3	3	3
E	0	1	2	2	3	3	3
D	0	1	2	2	3	4	4

```
Length = 4
```

```
The subsequence:   1)  ABAD      2)  ACAD
```

The length is the last cell of the table. We recover the letters by walking back from the last cell: when the two letters match we take the letter and move diagonally, otherwise we move to the larger of top and left. The coloured cells are the path of `ABAD`. When top and left are equal we may go either way, which is where the second answer `ACAD` comes from.

```
Check:   X = A C B A E D   →   A . B A . D   and   A C . A . D
```

```
         Y = A B C A D F   →   A B . A D .   and   A . C A D .
```

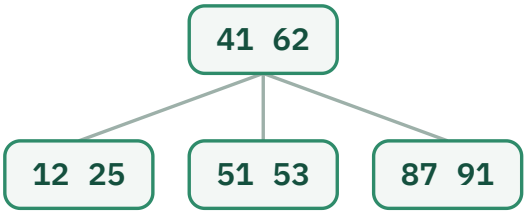
06 Solution Q6: B-tree

A B-tree of order 5: every node has at most 4 keys and at most 5 children. Every node other than the root has at least 2 keys. When a node reaches 5 keys it splits: the middle key moves

up to the parent, and the rest splits into two halves.

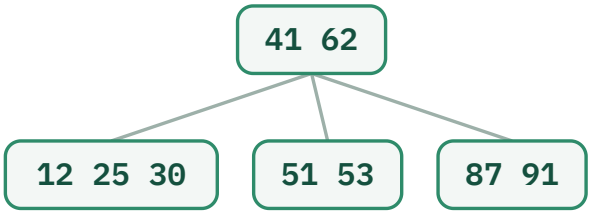
A Building the tree

```
Insert 25, 53, 91, 41      →   one node    [25, 41, 53, 91]
Insert 12                  →   5 keys, split, 41 moves up
Insert 87, 62              →   right leaf    [53, 62, 87, 91]
Insert 51                  →   5 keys, split, 62 moves up
```



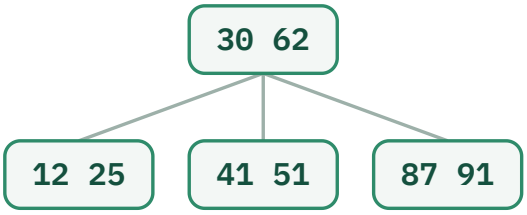
B Insert 30

`30` goes to the left leaf. It now has 3 keys, so there is no split.



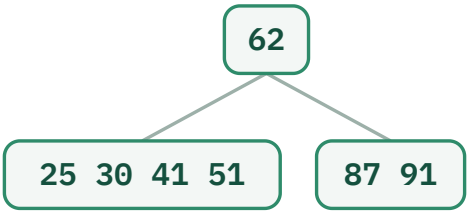
C1 Delete 53

The leaf of `53` is left with one key, below the minimum (underflow). The left sibling has 3 keys, so we borrow from it: its largest key moves up to the parent, and the parent's key moves down into the leaf.



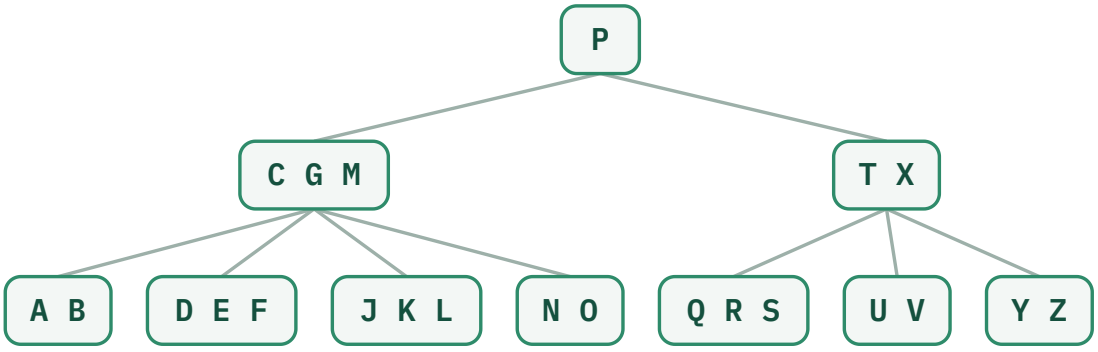
C2 Delete 12

The leaf of `12` is left with one key. Its sibling has only 2 keys, so we cannot borrow. We merge: the two leaves and the parent key between them become one node.

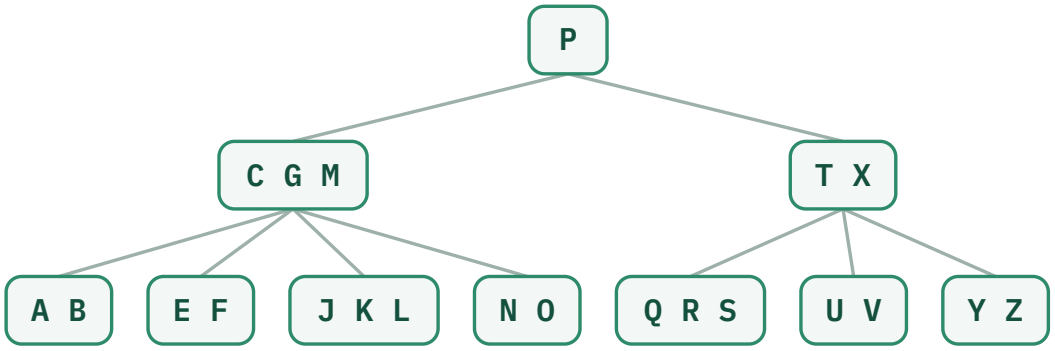


D Delete D, then delete X

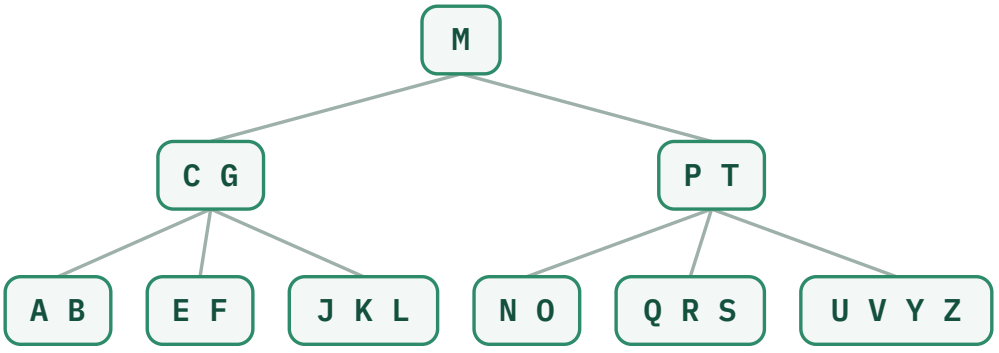
The figure in the photo is small and not fully readable. I assumed it is the well-known CLRS textbook figure, in which every node other than the root has at least 2 keys:



Delete D: the leaf `D E F` has 3 keys, so we remove D directly and `E F` remains.



Delete X: the key X is in the internal node `T X`, which has only the minimum. Before descending into it we strengthen it from its sibling `C G M`: P moves down into it, M moves up to the root, and the leaf `N O` moves across with it. Then neither child next to X can give a replacement (both `U V` and `Y Z` are at the minimum), so we merge them with X and remove X.



> This part is the least certain in the whole solution, because the figure is unclear and the deletion procedure differs between textbooks. Check it against the instructor's slide.